

Lab 3: Architectural Style Justification

Name: Udit Mourya SRN: PES2UG24CS565

Academic Elective Bidding & Allocation System

Architecture Selection

We chose Layered Architecture for the Academic Elective Bidding & Allocation System.

Reason 1: Clear separation for a semester-long, single-developer project

The system has three natural concerns — the student/registrar/instructor-facing interface, the bidding and allocation business logic, and persistent storage of electives, bids, and allocation records. Organizing these into Presentation, Business, and Data layers keeps each concern independently testable and modifiable without requiring the operational overhead of coordinating multiple independently-deployed services, which matters for a project built and maintained by a single developer across a semester.

Reason 2: Usage pattern is bursty and predictable, not continuously high-traffic

Load on this system spikes sharply during the bidding window and again briefly when the solver runs, rather than sustaining high concurrent traffic year-round. A layered monolith handles this pattern well without the added complexity microservices would introduce (service discovery, inter-service network calls, distributed data consistency) for a system that does not need to scale individual components independently.

Security Advantage

Centralizing all bid and allocation logic in a single Business layer means there is exactly one enforcement point for validating student bids and controlling access to the Database component — only the Bid & Allocation Manager Component is permitted to read or write bid, credit, and allocation records. This reduces the attack surface compared to an architecture where multiple independently-exposed services each have their own direct path to the data store.

Performance Benefit

Because the Business layer components run in-process, the Bid & Allocation Manager Component calls the Allocation Engine and Database components through direct in-memory calls rather than network requests. This avoids the inter-service latency a microservices approach would add between the solver and the datastore, which is important given the requirement that the allocation solver must resolve 5,000 student bids and schedule conflicts in under 30 seconds.